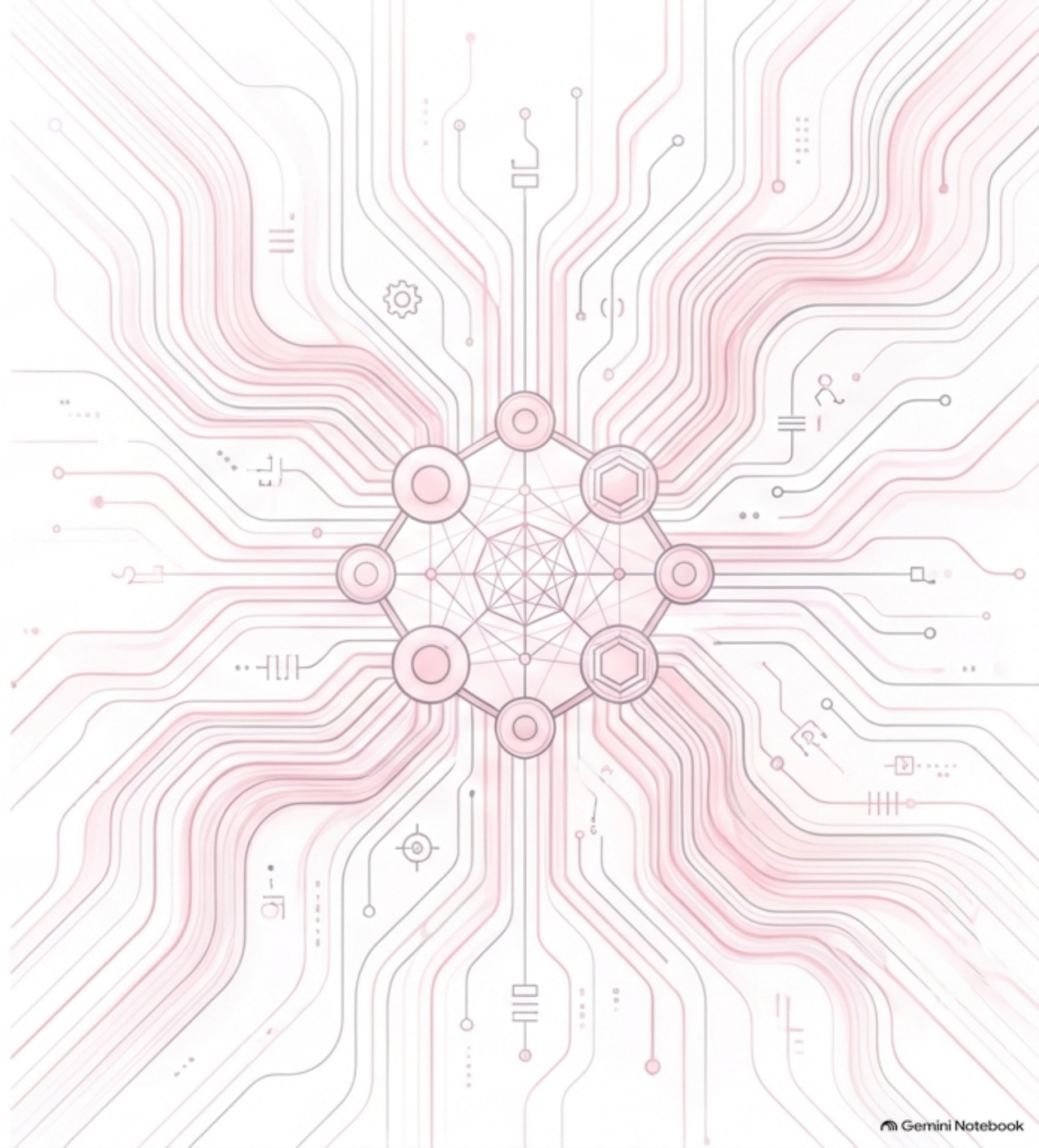


Depuradores y Emuladores para Sistemas Embebidos

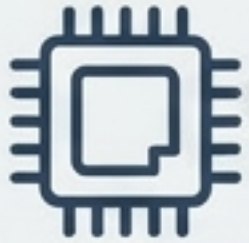
Caso QEMU + GDB

Curso: Taller de Sistemas Embebidos | Marycruz Fallas Barquero



La “caja negra” del hardware: ¿Por qué es tan difícil depurar?

Alta dependencia física



El software está amarrado a componentes físicos que pueden desgastarse o estar mal conectados.

Baja observabilidad



No hay pantallas, No hay pantallas, consolas ni terminales por defecto para ver qué ocurre internamente.

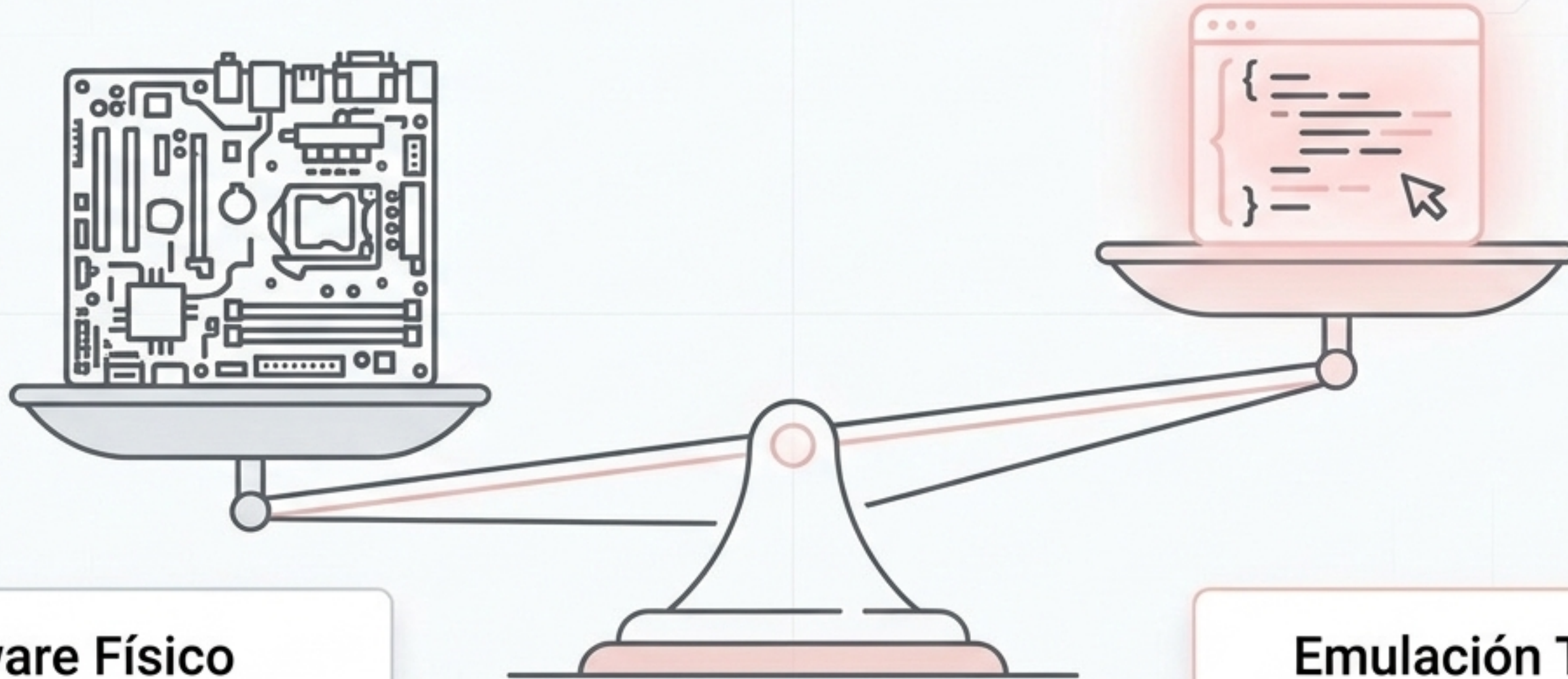
Dificultad de diagnóstico



Algunos errores aparecen bajo condiciones específicas y pueden ser difíciles de reproducir y analizar.

Solución: Entornos emulados para ‘abrir’ la caja negra y visualizar el interior.

El dilema del desarrollo: Validar software sin hardware



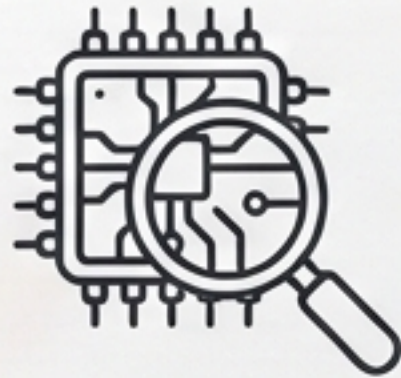
Hardware Físico

Configuraciones físicas complejas, cables cruzados y lentitud en la fabricación de la placa.

Emulación Temprana

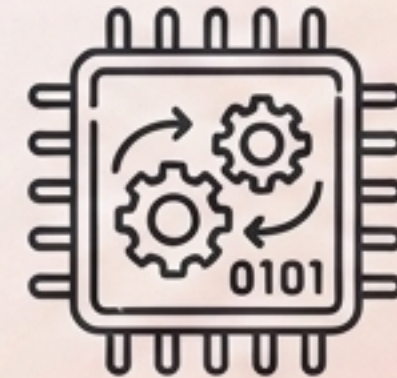
Necesidad de escribir, probar y validar el firmware mucho antes de que el hardware real sea fabricado.

Definiendo el vocabulario: Depuración y Emulación



Depuración

Inspeccionar, pausar y modificar el estado del software y hardware en tiempo real (registros, memoria).



Emulación

Representación mediante software de una arquitectura o sistema objetivo que permite ejecutar software diseñado para ese entorno.



Emulación $\neq$ Simulación. La simulación modela el comportamiento de un sistema; la emulación proporciona un entorno compatible con la arquitectura o sistema objetivo.

Separando roles: Host, Target y Depuración Cruzada

Host (Anfitrión)

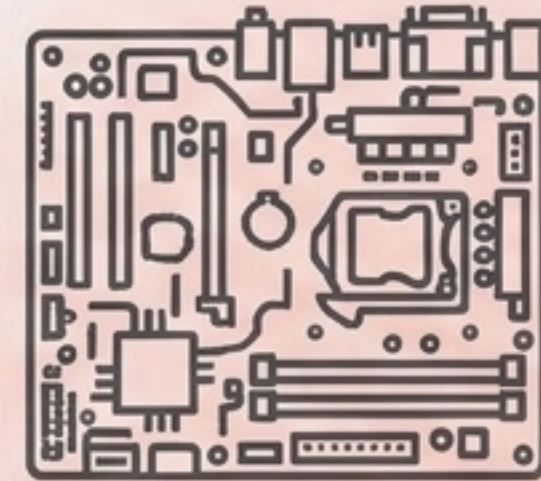


Equipo de desarrollo
(ej. Laptop x86_64).
Ejecuta el compilador y
el depurador (GDB).

Cross- debugging

Técnica para depurar desde
el Host una arquitectura
distinta ejecutándose en el
Target.

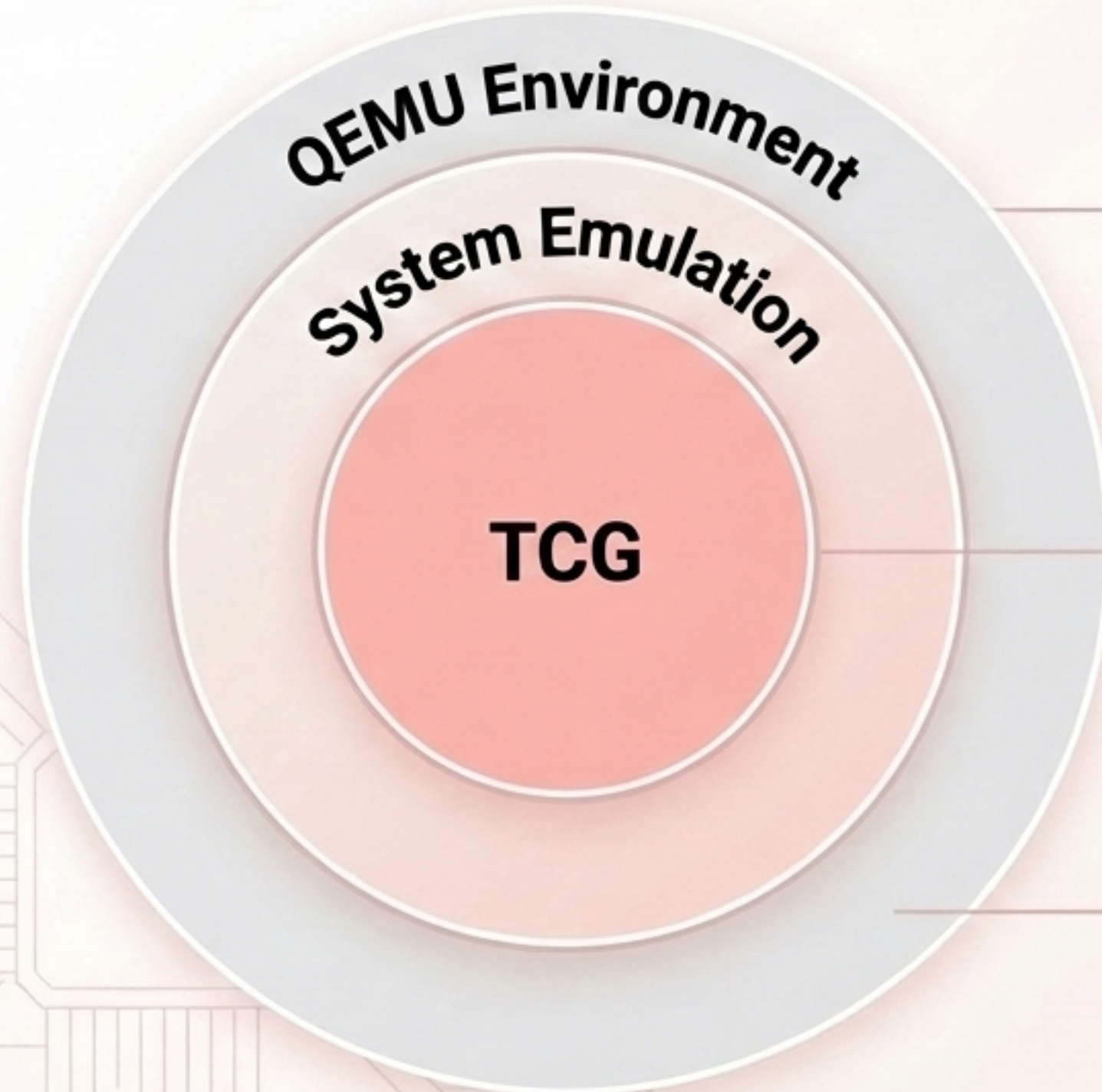
Target (Objetivo)



Arquitectura embebida
donde corre el programa
(ej. Placa ARM Cortex-M).

QEMU: El creador del entorno físico virtual

Emulador y virtualizador de código abierto multiplataforma para validación temprana de firmware.

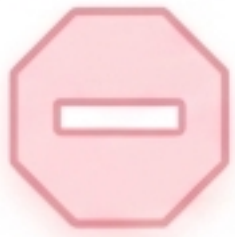


TCG (Tiny Code Generator): Motor dinámico que traduce instrucciones de la arquitectura Target para la CPU del Host.

System Emulation: Emula una plataforma que puede incluir procesador, memoria, buses y dispositivos modelados.

Rol en embebidos: Proporciona un Target virtual sobre el cual puede ejecutarse software bare-metal o un sistema operativo, según la máquina emulada.

GNU GDB: El inspector y controlador de ejecución



Breakpoints

Puntos de interrupción programados.



Paso a paso

Control granular del código.



Variables

Inspección de datos en vivo.



Memoria

Lectura directa de direcciones RAM.



Registros

Estado interno de la CPU.



Backtrace

Rastreo de la pila tras fallas.

La Pareja Perfecta: QEMU vs GDB

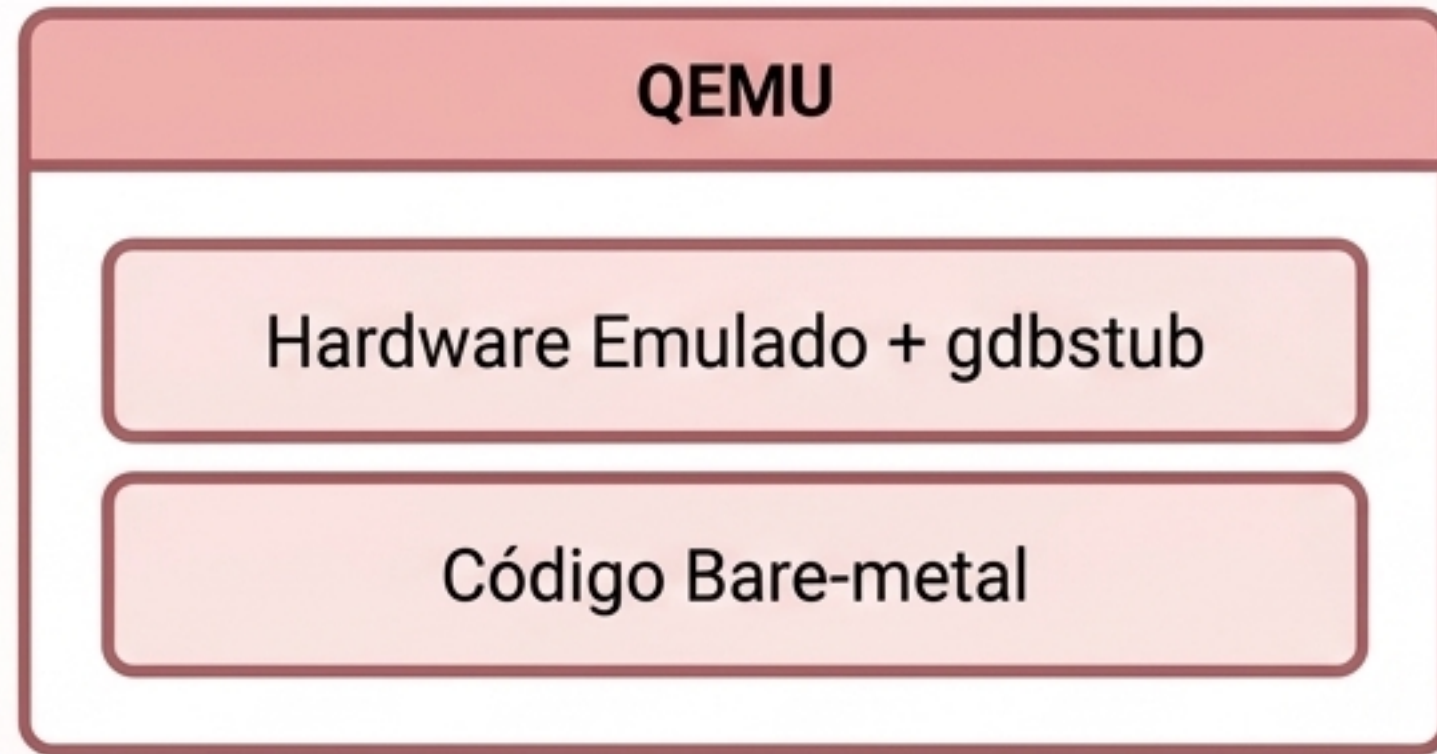
Atributo	QEMU	GDB
Rol principal	Entorno y Hardware Emulado 	Inspector de Software 
Qué hace	Ejecuta el software del Target y emula/modela la CPU, memoria y dispositivos soportados.	Pausa la ejecución, lee la memoria y evalúa el estado de las variables.
Símil didáctico	Es “el mundo físico” donde ocurre la acción.	Es “el control remoto y los rayos X”.

Remote Debugging: El puente entre dos mundos

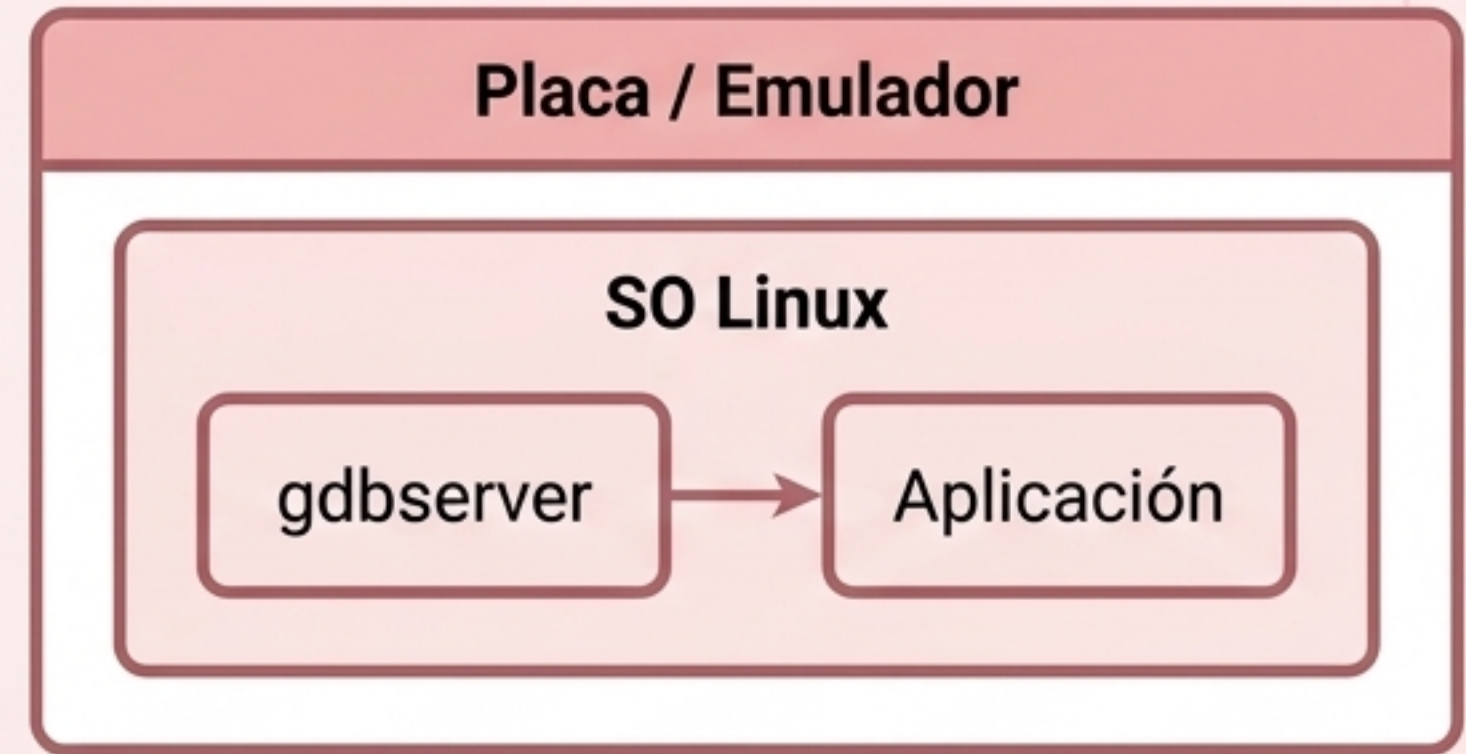


- **Concepto:** GDB se ejecuta en el Host mientras el programa se ejecuta en un Target real o emulado.
- **Comunicación:** GDB envía comandos de control e inspección hacia el Target mediante una interfaz de depuración remota.
- **Protocolo:** GDB Remote Serial Protocol puede utilizar TCP/IP o una conexión serial. En nuestro caso se utiliza localhost:1234.

Servidores de depuración: gdbstub vs gdbserver



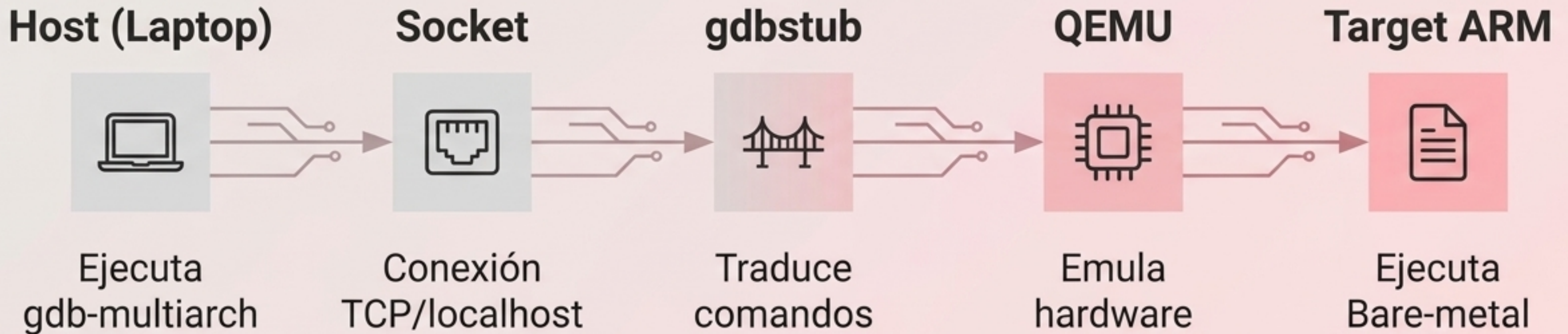
Servidor embebido directamente en el emulador. Detiene el hardware virtual por completo. Ideal para desarrollo bare-metal.



Aplicación independiente que corre dentro de un SO Target. Solo pausa la aplicación, no el hardware completo.

En nuestro mini-taller utilizaremos gdbstub para obtener control absoluto del hardware.

Arquitectura Completa del Flujo de Depuración



Símbolos de depuración: El mapa del código (Opción -g)

El procesador ejecuta instrucciones máquina asociadas a **direcciones de memoria**. La opción **-g** agrega información de depuración que permite a GDB relacionar esas direcciones con **funciones, variables, archivos y líneas del código fuente**.

GDB sin símbolos

0x0040004c in ??()

gcc -g

GDB con símbolos

main() at archivo.c:10

El vocabulario de GDB: Comandos Esenciales

Control

`break` (pausar)

`continue` (reanudar)

`bt` (backtrace/ruta)

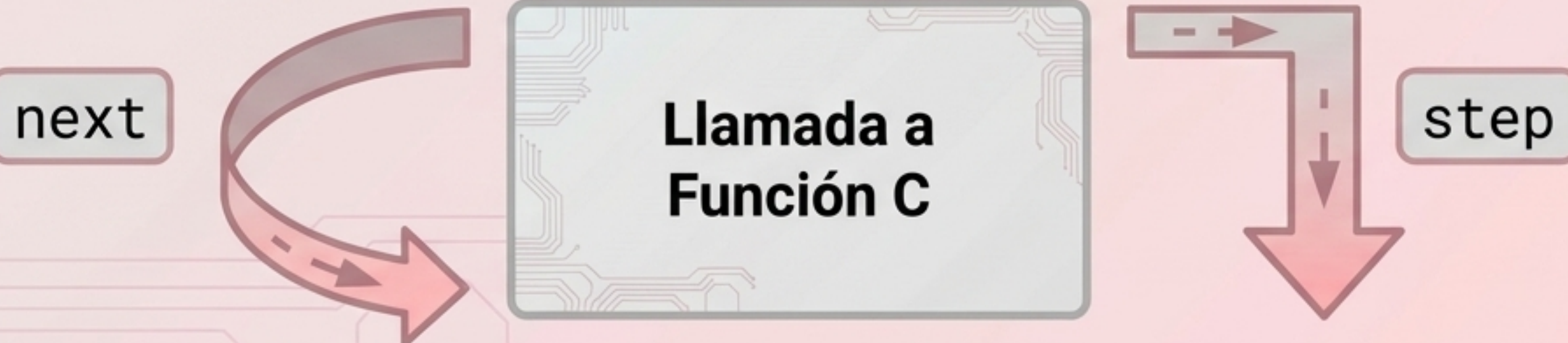
Inspección

`print` (ver variables)

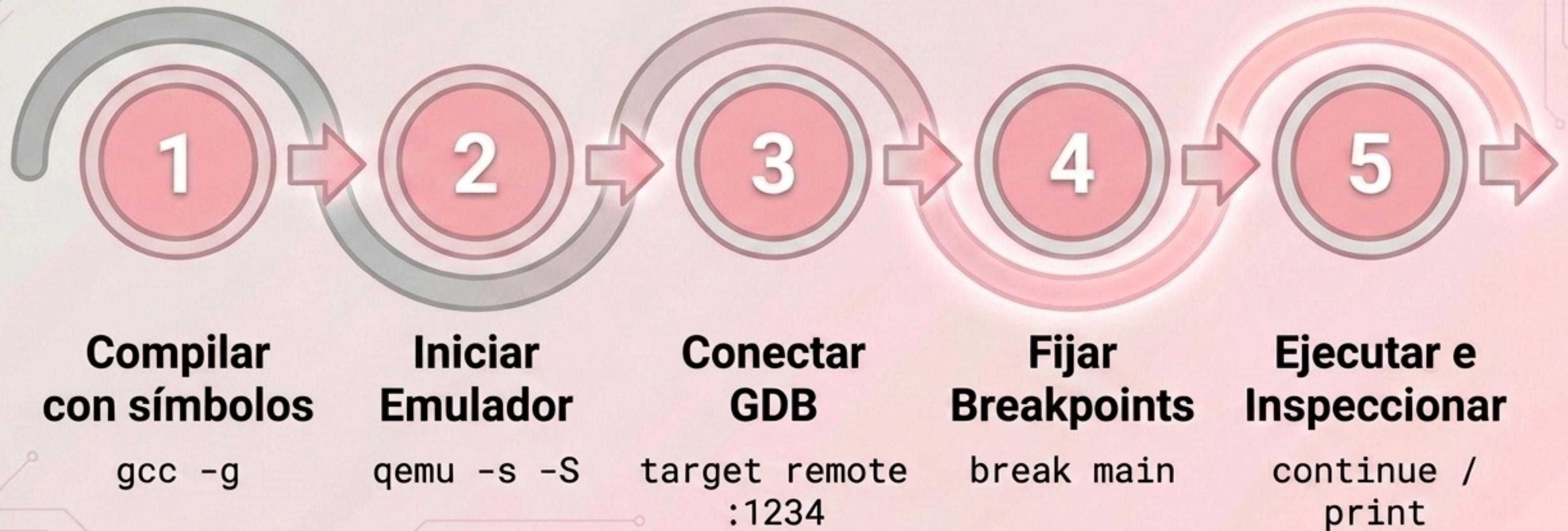
`info registers` (estado del CPU)

`x` (examinar memoria RAM)

Diferencia Clave: next vs step



Flujo de una sesión de depuración exitosa



De la academia a la industria: Aplicaciones reales



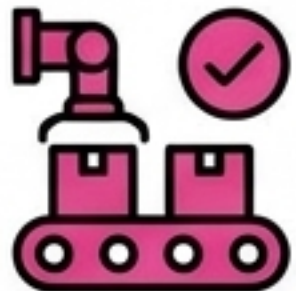
Desarrollo temprano

Desarrollar y probar software antes de disponer del hardware objetivo.



Depuración de bajo nivel

Inspeccionar firmware, registros, memoria y flujo de ejecución sobre un Target emulado.



Pruebas automatizadas / CI-CD

Ejecutar pruebas reproducibles sobre Targets emulados como parte de procesos automatizados.



Laboratorios virtuales

Proporcionar entornos equivalentes de práctica sin requerir una tarjeta física para cada estudiante.

La realidad técnica: Limitaciones frente al hardware físico

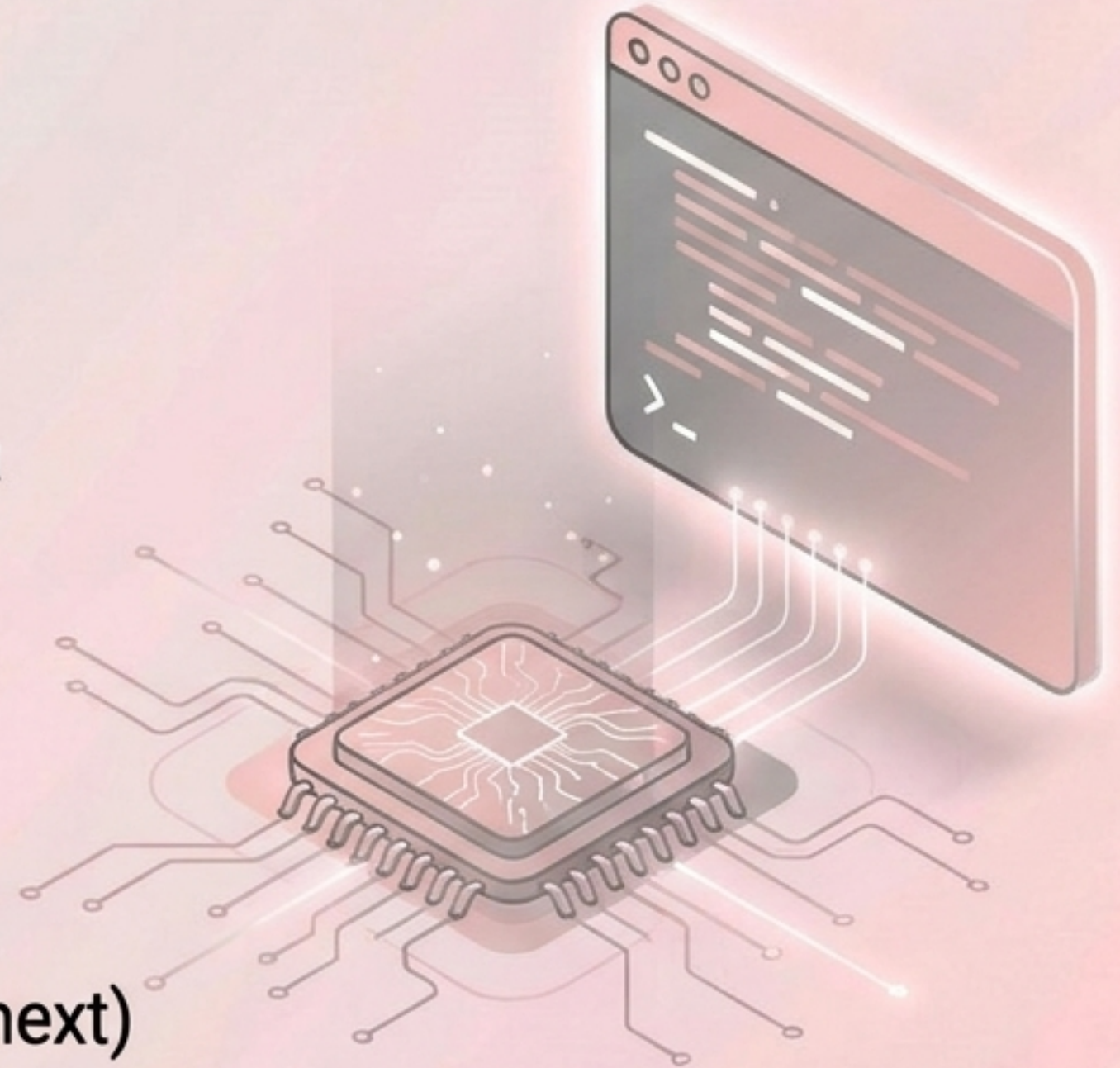


- **Temporización:** QEMU no necesariamente reproduce con exactitud los tiempos de ciclo y el comportamiento temporal del hardware físico.
- **Periféricos:** Solo pueden utilizarse los dispositivos y periféricos que estén modelados para la máquina emulada.
- **Validación en hardware:** La validación sobre hardware real sigue siendo necesaria cuando se deben comprobar temporización, periféricos físicos y comportamiento específico de la plataforma.

Nuestro Mini-Taller: Lo que vamos a construir

Laboratory Checklist

- ✓ Programa escrito en C.
- ✓ Compilación cruzada con símbolos de depuración (-g).
- ✓ Ejecución pura (Bare-metal) sin sistema operativo.
- ✓ Entorno Target ARM emulado con QEMU.
- ✓ Conexión remota desde el Host con gdb-multiarch usando gdbstub.
- ✓ Práctica viva: breakpoints, saltos (step/next) e inspección de registros de CPU.



Conclusiones y Referencias Técnicas

Ideas Clave

- La emulación permite ejecutar y observar software sobre un Target virtual sin depender inmediatamente del hardware físico.
- **GDB** permite controlar la ejecución e inspeccionar variables, memoria, registros y pila de llamadas.
- **Host** y **Target** pueden utilizar arquitecturas diferentes mediante cross-debugging.
- **QEMU** y **GDB** forman un entorno útil para desarrollo, depuración y aprendizaje de sistemas embebidos.

Referencias

QEMU Documentation – GDB usage

QEMU Documentation – Arm System Emulator / System Emulation

GNU GDB Manual – Debugging with GDB

GNU GDB Manual – Remote Debugging

Arm – How to debug: CoreSight basics